

Cómputo Concurrente 2026

Práctica 7

Implementaciones de Contadores: Balance entre Corrección y Eficiencia

Profa: Gilde Valeria Rodríguez

1 Contexto

Razonar sobre la corrección de un algoritmo concurrente es un desafío. Existen nociones de corrección que definen lo que es un comportamiento válido a partir de transformar una historia concurrente (con hilos ejecutándose al mismo tiempo) en una historia secuencial (como si ocurrieran uno tras otro) para analizar si el resultado tiene sentido.

Esta transformación sigue reglas que van de más a menos estrictas:

- **Linealizabilidad (Linearizability [HW90]):** Es el modelo más fuerte. Cada operación parece tener efecto de forma instantánea y atómica en un punto único en el *tiempo real*, ubicado estrictamente entre su invocación y su respuesta.
- **Consistencia Secuencial (Sequential Consistency [Lam79]):** Relaja el tiempo real. Exige que exista un orden global único de todas las operaciones que respete el *orden de programa* de cada hilo de forma individual.
- **Consistencia en la Inactividad (Quiescent Consistency [HS08]):** No impone un orden estricto entre operaciones concurrentes superpuestas. Solo garantiza el orden a través de los periodos de inactividad: si la operación A termina antes de que la operación B comience (separadas por un instante donde nadie hace nada), A precederá a B.
- **Consistencia Eventual (Eventual Consistency [Vog09]):** El modelo más relajado. Si cesan las actualizaciones, todas las réplicas del sistema convergerán *eventualmente* al mismo estado. Tolerar que diferentes hilos vean estados distintos temporalmente.

Los algoritmos concurrentes que garantizan linealizabilidad suelen imponer un **cuello de botella único** (como un candado global o una variable fuertemente sincronizada). Esto destruye la escalabilidad a medida que aumenta el número de hilos.

La solución propuesta en la literatura [Sha11] para alcanzar el máximo rendimiento es **relajar nuestras exigencias de corrección**. Al pasar de estructuras linealizables a estructuras con consistencia quiescente o eventual, permitimos que las operaciones se "dispersen" en el espacio de memoria, evitando que los hilos choquen entre sí y maximizando la eficiencia.

En los sistemas distribuidos masivos, la consistencia fuerte no solo es ineficiente, sino matemáticamente imposible frente a particiones de red (Teorema CAP [GL02]). Por ejemplo, en contadores escalables a nivel global, los mensajes se atrasan, se pierden o se reordenan. Exigir linealizabilidad requeriría detener internet por cada incremento. Por ello, se adopta la **Consistencia Eventual**: un cliente puede incrementar su contador local inmediatamente, y el protocolo se encarga de reconciliar estos valores en segundo plano para que, eventualmente, el total sea preciso [AB19].

2 Distintos Sabores de Contadores

A continuación, analizaremos tres implementaciones de contadores, desde la más estricta y lenta, hasta las más relajadas y rápidas.

2.1 Contador Linealizable

El enfoque clásico y seguro. Al delegar la operación al hardware subyacente mediante `incrementAndGet()`, garantizamos el nivel de corrección más alto.

```
1 import java.util.concurrent.atomic.AtomicLong;
2
3 public class LinearizableCounter {
4     private final AtomicLong count = new AtomicLong(0);
5
6     public void increment() {
7         count.incrementAndGet();
8     }
9
10    public long fetch() {
11        return count.get();
12    }
13 }
```

Listing 1: LinearizableCounter.

¿Por qué es Linealizable?

Porque la instrucción `incrementAndGet()` se ejecuta a nivel de procesador como una operación indivisible. Si el Hilo A y el Hilo B llaman al método al mismo tiempo, el hardware formará una fila instantánea en tiempo real.

Ejemplo de ejecución: Si A llama al método a las 10:00:01 y B a las 10:00:02, es matemáticamente imposible que B no vea el incremento de A. Al cumplir con el tiempo real, cumple automáticamente con todas las consistencias inferiores (Secuencial, Quiescente, etc.). El costo de esto es la alta contención: si 1,000 hilos llaman a `incrementAndGet()`, 999 tendrán que esperar su turno.

2.2 Sloppy Counter (Consistencia Eventual)

Este contador asigna memoria privada a cada hilo (`ThreadLocal`) y solo vuelca los datos a la variable global cuando supera un umbral. Es ultrarrápido, pero sacrifica la frescura de los datos.

```
1 public class SloppyCounter {
2     private final AtomicLong globalCount = new AtomicLong(0);
3     private final int threshold = 100;
4     private final ThreadLocal<Long> localCount = ThreadLocal.withInitial(() -> 0L);
5
6     public void increment() {
7         long current = localCount.get() + 1;
8         if (current >= threshold) {
9             globalCount.addAndGet(current);
10            localCount.set(0L);
11        } else {
12            localCount.set(current);
13        }
14    }
15
16    public long fetch() {
17        return globalCount.get(); // Lectura aproximada (eventual)
18    }
19 }
```

Listing 2: SloppyCounter.

¿Por qué es Consistente Eventualmente?

Porque si todos los hilos dejaran de incrementar y forzaran un volcado de sus variables locales, todos verían el mismo valor final correcto.

¿Por qué NO alcanza la Consistencia Secuencial?

Para ser secuencialmente consistente, el sistema global debe respetar el *orden de programa* de cada hilo.

Ejemplo de ejecución que falla:

1. El Hilo A realiza 50 incrementos. (Su código local avanza).

2. El Hilo A cambia una variable compartida: `bandera = true`.
3. El Hilo B espera a ver la bandera. Ve `bandera == true` y procede a llamar a `fetch()`.
4. El Hilo B lee `0` (porque A no alcanzó el umbral de 100 para volcar).

En una historia secuencial válida, si el Hilo B actuó *después* de ver la bandera del Hilo A, obligatoriamente debió ver los incrementos que A hizo *antes* de levantar la bandera. Como lee 0, se rompe el orden lógico.

2.3 Árbol de Difracción (Consistencia Quiescente)

Inspirado en Shavit, los hilos atraviesan un árbol binario mediante balanceadores, dividiendo el tráfico hasta llegar a múltiples hojas contadoras. Para leer el total, el método `fetch` recorre y suma todas las hojas.

```
1 import java.util.concurrent.atomic.AtomicInteger;
2 import java.util.concurrent.atomic.AtomicLong;
3
4 public class ShavitTreeCounter {
5
6     // Nodo interno (Balanceador) o Nodo Hoja (Contador)
7     static class Node {
8         boolean isLeaf;
9         Node left, right;
10        AtomicInteger balancer = new AtomicInteger(0); // Prisma de difraccion
11        AtomicLong count = new AtomicLong(0); // Contador real
12
13        Node() { this.isLeaf = true; }
14        Node(Node left, Node right) {
15            this.isLeaf = false;
16            this.left = left;
17            this.right = right;
18        }
19    }
20
21    private final Node root;
22
23    public ShavitTreeCounter(int depth) {
24        this.root = buildTree(depth);
25    }
26
27    private Node buildTree(int depth) {
28        if (depth == 0) return new Node();
29        return new Node(buildTree(depth - 1), buildTree(depth - 1));
30    }
31
32    public void increment() {
33        Node current = root;
34        while (!current.isLeaf) {
35            int route = current.balancer.getAndIncrement();
36            current = (route % 2 == 0) ? current.left : current.right;
37        }
38        current.count.incrementAndGet();
39    }
40
41    public long fetch() {
42        return sumLeaves(root);
43    }
44
45    private long sumLeaves(Node node) {
46        if (node.isLeaf) return node.count.get();
47        return sumLeaves(node.left) + sumLeaves(node.right);
48    }
49 }
```

Listing 3: Estructura base del ShavitTreeCounter.

¿Por qué garantiza Consistencia Quiescente?

Porque el contador solo garantiza devolver el valor exacto si *todas las operaciones de incremento se detienen por completo* (quiescence) antes de invocar el `fetch`.

Analogía del estadio: Es como una multitud entrando por 16 puertas distintas. Mientras la gente entra, es imposible contarlos exactamente sin detener el flujo. Solo cuando todos se sientan (inactividad/quiescence), cuentas y el número cuadra perfecto.

¿Por qué NO alcanza la Consistencia Secuencial?

Porque la lectura (`fetch`) no es atómica; viaja hoja por hoja mientras otros hilos siguen modificando el árbol.

Ejemplo de ejecución que falla (Ver el futuro y perderse el pasado):

1. Imagina un árbol con dos hojas: Izquierda(0) y Derecha(0).
2. Un Hilo Lector comienza a sumar y lee la hoja Izquierda (Valor: **0**).
3. Un Hilo Escritor entra, incrementa la Izquierda (+1) y luego incrementa la Derecha (+1). (Ambas quedan en 1).
4. El Hilo Lector avanza a la hoja Derecha, lee su valor (Valor: **1**) y reporta un Total de **1**.

El Lector vio el evento "futuro" (el incremento en la Derecha) pero se perdió el evento "pasado" (el incremento en la Izquierda). En una historia secuencial, si ves el paso 2 de un hilo, estás obligado a haber visto su paso 1. Aquí no ocurre, rompiendo la consistencia secuencial.

3 Ejercicios

Instrucciones:

- Entrega en un **PDF** las respuestas de los ejercicios teóricos. Para los que requieran código, añade una breve descripción de los programas entregados.
- Solo un integrante del equipo debe subir la práctica. Los demás deben marcar la tarea como entregada y escribir un comentario privado con el nombre de quien entregó.
- Todos los programas y el PDF deben ir en un **archivo ZIP** nombrado: **P5_ApellidoPaternoNombre**.
- Utilizar **Java 21 LTS**. Se penalizará si no compila en esta versión.

Tiempo estimado: ≈ 1.5 hr

Puntaje total: 100

1. Ejercicio 1: El Sloppy Counter Modificado

Se sugiere hacer una pequeña modificación al `SloppyCounter` para "mejorar" su consistencia. Propone que siempre que un hilo llame al método `fetch()`, el código primero **obligue a ese mismo hilo** a volcar su `localCount` actual al `globalCount`, y recién entonces retorne el valor global.

```
1 public long fetch() {
2     long currentLocal = localCount.get();
3     if (currentLocal > 0) {
4         globalCount.addAndGet(currentLocal);
5         localCount.set(0L);
6     }
7     return globalCount.get();
8 }
9
```

Listing 4: Modificación a `fetch()`

Analiza este escenario con múltiples hilos trabajando. ¿Esta modificación hace que la estructura sea Linealizable, Secuencialmente Consistente o se mantiene Eventualmente Consistente? Argumenta tu respuesta detallando qué sucede con el estado local de los *otros* hilos que no llamaron a `fetch()`.

2. Ejercicio 2: Lectura Libre de Obstrucciones en el Árbol

Como hemos visto, el `ShavitTreeCounter` original ofrece un método `fetch` que únicamente garantiza *Consistencia Quiescente*. Si intentamos volverlo Linealizable utilizando candados (*locks*), reintroducimos un cuello de botella masivo.

Para este inciso, implementarás una solución que recupere la linealizabilidad sin utilizar mecanismos de exclusión mutua.

- (a) **Diseña e implementa** una modificación del método `fetch` utilizando la técnica de **Double-Collect** (Doble Recolección) detallada en la Sección 4 del libro [HS08]. Tu implementación debe garantizar que el método sea simultáneamente **obstruction-free** y **linealizable**.
- (b) **Describe tu código** explicando paso a paso cómo tu algoritmo maneja la recolección del estado sin detener a los hilos escritores.
- (c) **Argumenta teóricamente:**
 - ¿Por qué tu implementación cumple con la propiedad de progreso *obstruction-free*? (Explica por qué no alcanza a ser *wait-free*).
 - ¿Por qué el valor retornado es estrictamente *linealizable*? (Identifica claramente el punto de linealización de tu lectura).

References

- [AB19] Paulo Sérgio Almeida and Carlos Baquero. Scalable eventually consistent counters over unreliable networks. *Distrib. Comput.*, 32(1):69–89, February 2019.
- [GL02] Seth Gilbert and Nancy Lynch. Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services. *SIGACT News*, 33(2):51–59, June 2002.
- [HS08] Maurice Herlihy and Nir Shavit. *The Art of Multiprocessor Programming*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 2008.
- [HW90] Maurice P. Herlihy and Jeannette M. Wing. Linearizability: a correctness condition for concurrent objects. *ACM Trans. Program. Lang. Syst.*, 12(3):463–492, July 1990.
- [Lam79] Lamport. How to make a multiprocessor computer that correctly executes multiprocess programs. *IEEE Transactions on Computers*, C-28(9):690–691, 1979.
- [Sha11] Nir Shavit. Data structures in the multicore age. *Commun. ACM*, 54(3):76–84, mar 2011.
- [Vog09] Werner Vogels. Eventually consistent. *Commun. ACM*, 52(1):40–44, January 2009.